

07. Transaction Management trong Spring Boot

7.1. Giới thiệu về Transaction Management

Transaction (giao dịch) là một đơn vị công việc logic bao gồm một hoặc nhiều thao tác cơ sở dữ liệu mà phải được thực hiện như một khối duy nhất. Tất cả các thao tác trong một transaction phải thành công hoàn toàn, hoặc nếu có bất kỳ thao tác nào thất bại, toàn bộ transaction sẽ được rollback (hoàn tác) để đảm bảo tính nhất quán của dữ liệu.

Spring Framework cung cấp một mô hình quản lý transaction mạnh mẽ và linh hoạt, hỗ trợ cả programmatic transaction management (quản lý transaction bằng code) và declarative transaction management (quản lý transaction bằng annotation). Spring Boot làm cho việc cấu hình transaction management trở nên đơn giản hơn nhiều thông qua auto-configuration.

7.1.1. Các tính chất ACID của Transaction

Một transaction hợp lệ phải đảm bảo các tính chất ACID:

- **Atomicity (Tính nguyên tử):** Tất cả các thao tác trong transaction phải được thực hiện hoàn toàn hoặc không được thực hiện gì cả. Không có trạng thái trung gian.
- **Consistency (Tính nhất quán):** Transaction phải đảm bảo rằng cơ sở dữ liệu chuyển từ một trạng thái nhất quán sang một trạng thái nhất quán khác.
- **Isolation (Tính cô lập):** Các transaction đồng thời không được ảnh hưởng lẫn nhau. Mỗi transaction phải hoạt động như thể nó là transaction duy nhất đang chạy.
- **Durability (Tính bền vững):** Sau khi transaction được commit, các thay đổi phải được lưu trữ vĩnh viễn, ngay cả khi hệ thống gặp sự cố.

7.2. Declarative Transaction Management với

@Transactional

Cách phổ biến nhất để quản lý transaction trong Spring Boot là sử dụng annotation `@Transactional`. Annotation này có thể được áp dụng ở cấp độ lớp hoặc phương thức.

7.2.1. Cấu hình cơ bản

Spring Boot tự động cấu hình transaction management khi phát hiện các dependency như `spring-boot-starter-data-jpa` hoặc `spring-boot-starter-jdbc`. Bạn chỉ cần thêm `@EnableTransactionManagement` vào configuration class (mặc dù điều này thường không cần thiết trong Spring Boot):

Java

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import
org.springframework.transaction.annotation.EnableTransactionManagement;

@SpringBootApplication
@EnableTransactionManagement // Thường không cần thiết trong Spring Boot
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

7.2.2. Sử dụng `@Transactional` cơ bản

Java

```
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class BankService {

    private final AccountRepository accountRepository;
    private final TransactionHistoryRepository transactionHistoryRepository;

    public BankService(AccountRepository accountRepository,
                       TransactionHistoryRepository
transactionHistoryRepository) {
        this.accountRepository = accountRepository;
        this.transactionHistoryRepository = transactionHistoryRepository;
    }
}
```

```

    }

    @Transactional
    public void transferMoney(Long fromAccountId, Long toAccountId, Double
amount) {
        // Lấy thông tin tài khoản
        Account fromAccount = accountRepository.findById(fromAccountId)
            .orElseThrow(() -> new AccountNotFoundException("From account not
found"));
        Account toAccount = accountRepository.findById(toAccountId)
            .orElseThrow(() -> new AccountNotFoundException("To account not
found"));

        // Kiểm tra số dư
        if (fromAccount.getBalance() < amount) {
            throw new InsufficientFundsException("Insufficient funds");
        }

        // Thực hiện chuyển tiền
        fromAccount.setBalance(fromAccount.getBalance() - amount);
        toAccount.setBalance(toAccount.getBalance() + amount);

        // Lưu thay đổi
        accountRepository.save(fromAccount);
        accountRepository.save(toAccount);

        // Ghi lịch sử giao dịch
        TransactionHistory history = new TransactionHistory(fromAccountId,
toAccountId, amount);
        transactionHistoryRepository.save(history);

        // Nếu có bất kỳ exception nào xảy ra, toàn bộ transaction sẽ được
rollback
    }

    @Transactional(readOnly = true)
    public Account getAccountById(Long id) {
        // Transaction chỉ đọc, tối ưu hiệu suất
        return accountRepository.findById(id)
            .orElseThrow(() -> new AccountNotFoundException("Account not
found"));
    }
}

```

7.2.3. Các thuộc tính của @Transactional

Java

```
import org.springframework.transaction.annotation.Isolation;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderService {

    @Transactional(
        propagation = Propagation.REQUIRED,      // Mức độ lan truyền
transaction
        isolation = Isolation.READ_COMMITTED,    // Mức độ cô lập
        timeout = 30,                            // Thời gian timeout (giây)
        readOnly = false,                        // Transaction có thể ghi
        rollbackFor = {Exception.class},        // Rollback cho các exception
này
        noRollbackFor = {IllegalArgumentException.class} // Không rollback
cho các exception này
    )
    public void processOrder(Order order) {
        // Logic xử lý đơn hàng
        validateOrder(order);
        updateInventory(order);
        createPayment(order);
        sendNotification(order);
    }
}
```

7.3. Transaction Propagation (Lan truyền Transaction)

Transaction Propagation định nghĩa cách các transaction lồng nhau hoạt động khi một phương thức có `@Transactional` gọi một phương thức khác cũng có `@Transactional`.

7.3.1. Các loại Propagation

Java

```
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderService {

    private final PaymentService paymentService;
```

```

private final InventoryService inventoryService;
private final NotificationService notificationService;

// Constructor injection...

@Transactional(propagation = Propagation.REQUIRED) // Mặc định
public void processOrder(Order order) {
    // Sử dụng transaction hiện tại hoặc tạo mới nếu chưa có
    updateOrderStatus(order, "PROCESSING");

    paymentService.processPayment(order); // Sử dụng cùng transaction
    inventoryService.updateStock(order); // Sử dụng cùng transaction

    updateOrderStatus(order, "COMPLETED");
}

@Transactional(propagation = Propagation.REQUIRES_NEW)
public void logOrderEvent(Long orderId, String event) {
    // Luôn tạo transaction mới, tạm dừng transaction hiện tại
    OrderLog log = new OrderLog(orderId, event, LocalDateTime.now());
    orderLogRepository.save(log);
    // Transaction này độc lập với transaction gọi nó
}

@Transactional(propagation = Propagation.SUPPORTS)
public Order getOrderDetails(Long orderId) {
    // Sử dụng transaction hiện tại nếu có, không tạo mới nếu không có
    return orderRepository.findById(orderId).orElse(null);
}

@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void sendEmailNotification(String email, String message) {
    // Tạm dừng transaction hiện tại và thực hiện không có transaction
    emailService.send(email, message);
}

@Transactional(propagation = Propagation.NEVER)
public void validateOrderData(Order order) {
    // Ném exception nếu có transaction hiện tại
    if (order.getAmount() <= 0) {
        throw new IllegalArgumentException("Invalid order amount");
    }
}

@Transactional(propagation = Propagation.MANDATORY)
public void updateOrderInTransaction(Order order) {
    // Ném exception nếu không có transaction hiện tại
    orderRepository.save(order);
}

```

```
}  
}
```

7.3.2. Ví dụ thực tế về Propagation

Java

```
@Service  
public class ECommerceService {  
  
    private final OrderService orderService;  
    private final AuditService auditService;  
  
    @Transactional  
    public void placeOrder(Order order) {  
        try {  
            // Transaction chính  
            orderService.processOrder(order); // REQUIRED - sử dụng cùng  
transaction  
  
            // Ghi audit log - sử dụng transaction riêng  
            auditService.logOrderPlaced(order.getId()); // REQUIRES_NEW  
  
        } catch (PaymentException e) {  
            // Nếu payment thất bại, order sẽ được rollback  
            // Nhưng audit log vẫn được giữ lại vì nó dùng transaction riêng  
            throw e;  
        }  
    }  
}  
  
@Service  
public class AuditService {  
  
    @Transactional(propagation = Propagation.REQUIRES_NEW)  
    public void logOrderPlaced(Long orderId) {  
        // Transaction này độc lập, sẽ không bị rollback khi transaction  
chính thất bại  
        AuditLog log = new AuditLog("ORDER_PLACED", orderId,  
LocalDateTime.now());  
        auditLogRepository.save(log);  
    }  
}
```

7.4. Transaction Isolation Levels (Mức độ cô lập)

Isolation levels định nghĩa mức độ cô lập giữa các transaction đồng thời, ảnh hưởng đến hiệu suất và tính nhất quán của dữ liệu.

Java

```
import org.springframework.transaction.annotation.Isolation;
import org.springframework.transaction.annotation.Transactional;

@Service
public class ReportService {

    @Transactional(isolation = Isolation.READ_UNCOMMITTED)
    public void generateDraftReport() {
        // Có thể đọc dữ liệu chưa được commit (dirty read)
        // Hiệu suất cao nhất nhưng ít nhất quán nhất
    }

    @Transactional(isolation = Isolation.READ_COMMITTED) // Mặc định trong
    hầu hết RDBMS
    public void generateStandardReport() {
        // Chỉ đọc dữ liệu đã được commit
        // Tránh dirty read nhưng có thể có non-repeatable read
    }

    @Transactional(isolation = Isolation.REPEATABLE_READ)
    public void generateConsistentReport() {
        // Đảm bảo đọc cùng dữ liệu trong suốt transaction
        // Tránh dirty read và non-repeatable read nhưng có thể có phantom
        read
    }

    @Transactional(isolation = Isolation.SERIALIZABLE)
    public void generateCriticalReport() {
        // Mức cô lập cao nhất, tránh tất cả các vấn đề đồng thời
        // Hiệu suất thấp nhất do lock nhiều
    }
}
```

7.5. Programmatic Transaction Management

Trong một số trường hợp, bạn cần kiểm soát transaction một cách chi tiết hơn:

Java

```
import org.springframework.stereotype.Service;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.TransactionDefinition;
import org.springframework.transaction.TransactionStatus;
import org.springframework.transaction.support.DefaultTransactionDefinition;

@Service
public class ComplexOrderService {

    private final PlatformTransactionManager transactionManager;
    private final OrderRepository orderRepository;
    private final PaymentService paymentService;

    public ComplexOrderService(PlatformTransactionManager transactionManager,
                               OrderRepository orderRepository,
                               PaymentService paymentService) {
        this.transactionManager = transactionManager;
        this.orderRepository = orderRepository;
        this.paymentService = paymentService;
    }

    public void processComplexOrder(Order order) {
        DefaultTransactionDefinition def = new
DefaultTransactionDefinition();

        def.setPropagationBehavior(TransactionDefinition.PROPGATION_REQUIRED);

        def.setIsolationLevel(TransactionDefinition.ISOLATION_READ_COMMITTED);
        def.setTimeout(30);

        TransactionStatus status = transactionManager.getTransaction(def);

        try {
            // Xử lý đơn hàng
            order.setStatus("PROCESSING");
            orderRepository.save(order);

            // Xử lý thanh toán
            PaymentResult result = paymentService.processPayment(order);

            if (result.isSuccessful()) {
                order.setStatus("PAID");
                orderRepository.save(order);
                transactionManager.commit(status);
            } else {
                // Rollback thủ công
            }
        }
    }
}
```



```

        transactionManager.rollback(status);
        throw new PaymentFailedException("Payment failed: " +
result.getErrorMessage());
    }

    } catch (Exception e) {
        transactionManager.rollback(status);
        throw e;
    }
}
}

```

7.6. Transaction với Multiple DataSources

Khi ứng dụng sử dụng nhiều database, bạn cần cấu hình transaction manager riêng:

Java

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import org.springframework.orm.jpa.JpaTransactionManager;
import org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean;
import org.springframework.transaction.PlatformTransactionManager;

import javax.persistence.EntityManagerFactory;
import javax.sql.DataSource;

@Configuration
public class DatabaseConfig {

    @Primary
    @Bean(name = "primaryTransactionManager")
    public PlatformTransactionManager primaryTransactionManager(
        @Qualifier("primaryEntityManagerFactory") EntityManagerFactory
entityManagerFactory) {
        return new JpaTransactionManager(entityManagerFactory);
    }

    @Bean(name = "secondaryTransactionManager")
    public PlatformTransactionManager secondaryTransactionManager(
        @Qualifier("secondaryEntityManagerFactory") EntityManagerFactory
entityManagerFactory) {
        return new JpaTransactionManager(entityManagerFactory);
    }
}

```

```

    }
}

@Service
public class MultiDatabaseService {

    @Transactional("primaryTransactionManager")
    public void saveToMainDatabase(User user) {
        userRepository.save(user);
    }

    @Transactional("secondaryTransactionManager")
    public void saveToAuditDatabase(AuditLog log) {
        auditLogRepository.save(log);
    }

    @Transactional("primaryTransactionManager")
    public void transferUserData(Long userId) {
        User user = userRepository.findById(userId).orElseThrow();

        // Sử dụng transaction manager khác cho audit
        auditService.logUserTransfer(userId);

        userRepository.delete(user);
    }
}

```

7.7. Best Practices cho Transaction Management

7.7.1. Các nguyên tắc tốt

Java

```

@Service
public class BestPracticeService {

    // ✅ Đặt @Transactional ở service layer, không phải repository layer
    @Transactional
    public void businessOperation() {
        // Logic nghiệp vụ
    }

    // ✅ Sử dụng readOnly cho các operation chỉ đọc
    @Transactional(readOnly = true)
    public List<Product> getAllProducts() {

```

```

        return productRepository.findAll();
    }

    // ✅ Xử lý exception cụ thể
    @Transactional(rollbackFor = {BusinessException.class,
DataException.class})
    public void criticalOperation() {
        // Logic quan trọng
    }

    // ✅ Tách các operation độc lập
    @Transactional
    public void processOrder(Order order) {
        validateOrder(order);
        saveOrder(order);

        // Gửi email trong transaction riêng để không ảnh hưởng đến business
logic
        notificationService.sendOrderConfirmation(order.getId());
    }
}

@Service
public class NotificationService {

    // Transaction riêng cho notification
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void sendOrderConfirmation(Long orderId) {
        // Gửi email
    }
}

```

7.7.2. Các lỗi thường gặp

Java

```

@Service
public class CommonMistakesService {

    // ❌ Không nên đặt @Transactional trên private method
    @Transactional
    private void privateMethod() {
        // Spring AOP không thể intercept private method
    }

    // ❌ Self-invocation không hoạt động với @Transactional

```

```

public void publicMethod() {
    this.transactionalMethod(); // Transaction sẽ không hoạt động
}

@Transactional
public void transactionalMethod() {
    // Logic
}

// ✅ Cách đúng: inject self hoặc sử dụng ApplicationContext
@Autowired
private ApplicationContext applicationContext;

public void correctPublicMethod() {
    CommonMistakesService self =
applicationContext.getBean(CommonMistakesService.class);
    self.transactionalMethod(); // Transaction sẽ hoạt động
}
}

```

7.8. Bản chất của Transaction Management trong Spring Boot

Bản chất của Transaction Management trong Spring Boot là cung cấp một cách tiếp cận khai báo (declarative) và mạnh mẽ để quản lý tính nhất quán dữ liệu trong các ứng dụng. Thông qua annotation `@Transactional` và cơ chế AOP (Aspect-Oriented Programming), Spring Boot cho phép các nhà phát triển định nghĩa ranh giới transaction một cách rõ ràng mà không cần viết boilerplate code phức tạp. Hệ thống này đảm bảo tính ACID của các giao dịch, hỗ trợ nhiều mức độ cô lập và lan truyền transaction khác nhau, và tích hợp liền mạch với các công nghệ persistence như JPA, Hibernate, và JDBC. Điều quan trọng là Spring Boot tự động cấu hình transaction management dựa trên các dependency có trong classpath, giúp giảm thiểu cấu hình thủ công và cho phép các nhà phát triển tập trung vào logic nghiệp vụ thay vì các chi tiết kỹ thuật của quản lý transaction.

7.8. Rollback Rules (Quy tắc Rollback)

Theo mặc định, Spring Framework sẽ rollback một transaction khi một `RuntimeException` (hoặc bất kỳ subclass nào của nó) được ném ra. Ngược lại, nó sẽ **không** rollback transaction

khi một `Checked Exception` được ném ra. Điều này dựa trên giả định rằng `RuntimeException` thường chỉ ra một lỗi lập trình hoặc lỗi hệ thống không thể phục hồi, trong khi `Checked Exception` thường chỉ ra một điều kiện nghiệp vụ mà ứng dụng có thể xử lý.

Tuy nhiên, bạn có thể tùy chỉnh hành vi rollback này bằng cách sử dụng các thuộc tính `rollbackFor` và `noRollbackFor` của annotation `@Transactional`.

7.8.1. `rollbackFor`

Thuộc tính `rollbackFor` cho phép bạn chỉ định một hoặc nhiều loại ngoại lệ mà khi chúng được ném ra, transaction sẽ bị rollback, bất kể chúng là checked hay unchecked exceptions.

Cú pháp:

Java

```
@Transactional(rollbackFor = {Exception.class,
CustomBusinessException.class})
public void doSomethingThatMightThrowException() throws
CustomBusinessException {
    // ... logic ...
    if (someCondition) {
        throw new CustomBusinessException("Business rule violated");
    }
}
```

- `Exception.class` : Nếu bạn muốn rollback cho tất cả các loại `Exception` (bao gồm cả checked và unchecked), bạn có thể chỉ định `Exception.class`. Điều này thường được sử dụng khi bạn muốn một hành vi rollback mạnh mẽ hơn mặc định.
- `CustomBusinessException.class` : Nếu `CustomBusinessException` là một checked exception, việc chỉ định nó ở đây sẽ đảm bảo transaction được rollback khi ngoại lệ này xảy ra.

7.8.2. `noRollbackFor`

Thuộc tính `noRollbackFor` cho phép bạn chỉ định một hoặc nhiều loại ngoại lệ mà khi chúng được ném ra, transaction sẽ **không** bị rollback, ngay cả khi chúng là `RuntimeException`.

Cú pháp:

Java

```
@Transactional(noRollbackFor = {UserNotFoundException.class})
public void processUserRequest() {
    // ... logic ...
    if (userDoesNotExist) {
        throw new UserNotFoundException("User not found, but don't rollback transaction");
    }
    // ... logic tiếp theo, có thể là ghi log thành công
}
```

- `UserNotFoundException.class` : Nếu `UserNotFoundException` là một `RuntimeException`, việc chỉ định nó ở đây sẽ ngăn transaction bị rollback khi ngoại lệ này xảy ra. Điều này hữu ích trong các trường hợp bạn muốn ghi nhận một sự kiện (ví dụ: log lỗi) nhưng không muốn hoàn tác các thay đổi khác trong transaction.

7.8.3. Kết hợp `rollbackFor` và `noRollbackFor`

Bạn có thể sử dụng cả hai thuộc tính cùng lúc. Tuy nhiên, cần lưu ý rằng `noRollbackFor` có ưu tiên cao hơn `rollbackFor`.

Ví dụ:

Java

```
@Transactional(rollbackFor = Exception.class, noRollbackFor =
{DataIntegrityViolationException.class})
public void saveCriticalData() throws Exception {
    // Logic lưu dữ liệu
    // Nếu có bất kỳ Exception nào xảy ra, transaction sẽ rollback, TRỪ
    DataIntegrityViolationException
}
```

Trong ví dụ này, nếu `DataIntegrityViolationException` (là một `RuntimeException`) xảy ra, transaction sẽ không bị rollback, mặc dù `rollbackFor = Exception.class` đã được chỉ định. Điều này có thể hữu ích nếu bạn muốn xử lý lỗi vi phạm ràng buộc dữ liệu mà không ảnh hưởng đến các thao tác khác trong cùng một transaction (ví dụ: bạn muốn ghi log lỗi và tiếp tục xử lý các bản ghi khác).

7.8.4. Ví dụ thực tế

Giả sử bạn có một dịch vụ xử lý đơn hàng và bạn muốn rollback transaction nếu có lỗi hệ thống hoặc lỗi nghiệp vụ nghiêm trọng, nhưng không muốn rollback nếu một `ProductOutOfStockException` xảy ra (vì bạn muốn ghi log sự kiện này và có thể xử lý nó ở tầng cao hơn mà không làm mất các thay đổi khác).

Java

```
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

// Định nghĩa Custom Exception
class ProductOutOfStockException extends RuntimeException {
    public ProductOutOfStockException(String message) {
        super(message);
    }
}

class OrderProcessingException extends RuntimeException {
    public OrderProcessingException(String message) {
        super(message);
    }
}

@Service
public class OrderService {

    private final ProductRepository productRepository;
    private final OrderRepository orderRepository;
    private final NotificationService notificationService;

    public OrderService(ProductRepository productRepository, OrderRepository
orderRepository, NotificationService notificationService) {
        this.productRepository = productRepository;
        this.orderRepository = orderRepository;
        this.notificationService = notificationService;
    }

    @Transactional(rollbackFor = OrderProcessingException.class,
noRollbackFor = ProductOutOfStockException.class)
    public Order placeOrder(Long productId, int quantity) {
        Product product = productRepository.findById(productId)
            .orElseThrow(() -> new OrderProcessingException("Product not
found"));

        if (product.getStock() < quantity) {
            // Ném ngoại lệ này sẽ KHÔNG rollback transaction
        }
    }
}
```

```

        throw new ProductOutOfStockException("Product " +
product.getName() + " is out of stock.");
    }

    // Giảm số lượng sản phẩm
    product.setStock(product.getStock() - quantity);
    productRepository.save(product);

    // Tạo đơn hàng
    Order order = new Order(productId, quantity, product.getPrice() *
quantity);
    orderRepository.save(order);

    // Gửi thông báo (có thể trong transaction riêng)
    notificationService.sendOrderConfirmation(order.getId());

    // Nếu có lỗi khác xảy ra (ví dụ: database down,
    OrderProcessingException), transaction sẽ rollback
    return order;
}
}

@Service
public class NotificationService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void sendOrderConfirmation(Long orderId) {
        // Logic gửi email/SMS
        System.out.println("Sending order confirmation for order: " +
orderId);
    }
}

```

Trong ví dụ trên, nếu `ProductOutOfStockException` được ném ra, các thay đổi đã thực hiện (ví dụ: giảm số lượng sản phẩm) sẽ không bị rollback. Điều này có thể không phải là hành vi mong muốn trong mọi trường hợp, nhưng nó minh họa cách bạn có thể kiểm soát chi tiết quy tắc rollback. Thông thường, bạn sẽ muốn rollback mọi thứ nếu có lỗi nghiệp vụ.

Lưu ý quan trọng: Việc sử dụng `noRollbackFor` cần được cân nhắc kỹ lưỡng, vì nó có thể dẫn đến trạng thái dữ liệu không nhất quán nếu không được quản lý cẩn thận. Trong hầu hết các trường hợp, bạn sẽ muốn transaction rollback hoàn toàn khi có bất kỳ lỗi nào xảy ra trong logic nghiệp vụ.

7.9. TransactionalEventListener

`TransactionalEventListener` là một annotation trong Spring Framework cho phép bạn lắng nghe các sự kiện (events) được publish trong một transaction. Điều này rất hữu ích khi bạn muốn thực hiện một hành động nào đó (ví dụ: gửi email, cập nhật cache, ghi log audit) chỉ khi transaction chính đã thành công (commit) hoặc thất bại (rollback).

7.9.1. Các pha của `TransactionalEventListener`

`TransactionalEventListener` có thuộc tính `phase` để chỉ định khi nào listener sẽ được kích hoạt:

- **`AFTER_COMMIT` (mặc định):** Kích hoạt sau khi transaction đã được commit thành công. Đây là pha phổ biến nhất, đảm bảo rằng các hành động phụ thuộc vào dữ liệu đã được lưu trữ vĩnh viễn.
- **`AFTER_ROLLBACK` :** Kích hoạt sau khi transaction đã bị rollback. Hữu ích cho việc xử lý lỗi hoặc dọn dẹp tài nguyên.
- **`AFTER_COMPLETION` :** Kích hoạt sau khi transaction đã hoàn thành (commit hoặc rollback). Bạn có thể kiểm tra trạng thái của transaction trong listener.
- **`BEFORE_COMMIT` :** Kích hoạt ngay trước khi transaction được commit. Điều này cho phép bạn thực hiện các kiểm tra cuối cùng hoặc chuẩn bị dữ liệu trước khi commit.

7.9.2. Cách sử dụng `TransactionalEventListener`

Bước 1: Định nghĩa một Custom Event

Java

```
import org.springframework.context.ApplicationEvent;

public class OrderPlacedEvent extends ApplicationEvent {
    private final Long orderId;
    private final String customerEmail;

    public OrderPlacedEvent(Object source, Long orderId, String
customerEmail) {
        super(source);
        this.orderId = orderId;
        this.customerEmail = customerEmail;
    }
}
```

```

    public Long getOrderId() {
        return orderId;
    }

    public String getCustomerEmail() {
        return customerEmail;
    }
}

```

Bước 2: Publish Event từ Service

Java

```

import org.springframework.context.ApplicationEventPublisher;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final ApplicationEventPublisher eventPublisher;

    public OrderService(OrderRepository orderRepository,
ApplicationEventPublisher eventPublisher) {
        this.orderRepository = orderRepository;
        this.eventPublisher = eventPublisher;
    }

    @Transactional
    public Order createOrder(Order order) {
        Order savedOrder = orderRepository.save(order);
        // Publish event sau khi order được lưu vào DB (trong cùng
transaction)
        eventPublisher.publishEvent(new OrderPlacedEvent(this,
savedOrder.getId(), savedOrder.getCustomerEmail()));
        return savedOrder;
    }

    @Transactional
    public void cancelOrder(Long orderId) {
        Order order = orderRepository.findById(orderId)
            .orElseThrow(() -> new RuntimeException("Order not found"));
        order.setStatus("CANCELLED");
        orderRepository.save(order);
        eventPublisher.publishEvent(new OrderCancelledEvent(this, orderId,

```

```
order.getCustomerEmail()));  
    }  
}
```

Bước 3: Tạo Listener với `TransactionalEventListener`

Java

```
import org.springframework.stereotype.Component;  
import org.springframework.transaction.event.TransactionalEventListener;  
import org.springframework.transaction.event.TransactionPhase;  
  
@Component  
public class OrderEventListener {  
  
    // Lắng nghe sự kiện OrderPlacedEvent sau khi transaction commit  
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)  
    public void handleOrderPlacedEvent(OrderPlacedEvent event) {  
        System.out.println("Order Placed Event received (AFTER_COMMIT): Order  
ID " + event.getOrderId() + ", Customer Email: " + event.getCustomerEmail());  
        // Logic gửi email xác nhận, cập nhật hệ thống khác, v.v.  
        // Đảm bảo rằng order đã được lưu vào database trước khi thực hiện  
các hành động này  
    }  
  
    // Lắng nghe sự kiện OrderCancelledEvent sau khi transaction rollback  
    @TransactionalEventListener(phase = TransactionPhase.AFTER_ROLLBACK)  
    public void handleOrderCancelledEventRollback(OrderCancelledEvent event)  
    {  
        System.out.println("Order Cancelled Event received (AFTER_ROLLBACK):  
Order ID " + event.getOrderId() + ", Customer Email: " +  
event.getCustomerEmail());  
        // Logic xử lý khi hủy đơn hàng thất bại (ví dụ: hoàn tác các thay  
đổi bên ngoài transaction)  
    }  
  
    // Lắng nghe sự kiện OrderCancelledEvent sau khi transaction hoàn thành  
(commit hoặc rollback)  
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMPLETION)  
    public void handleOrderCancelledEventCompletion(OrderCancelledEvent  
event) {  
        System.out.println("Order Cancelled Event received  
(AFTER_COMPLETION): Order ID " + event.getOrderId() + ", Customer Email: " +  
event.getCustomerEmail());  
        // Bạn có thể kiểm tra event.getTxStatus() để biết trạng thái  
commit/rollback  
    }  
}
```

```
// Lắng nghe sự kiện OrderPlacedEvent ngay trước khi transaction commit
@TransactionalEventListener(phase = TransactionPhase.BEFORE_COMMIT)
public void handleOrderPlacedEventBeforeCommit(OrderPlacedEvent event) {
    System.out.println("Order Placed Event received (BEFORE_COMMIT):
    Order ID " + event.getOrderId() + ", Customer Email: " +
    event.getCustomerEmail());
    // Logic kiểm tra cuối cùng trước khi commit
}
}
```

7.9.3. Lợi ích của TransactionEventListener

- **Đảm bảo tính nhất quán:** Các hành động phụ thuộc vào kết quả của transaction chỉ được thực hiện khi transaction đã thành công hoặc thất bại theo đúng ý muốn.
- **Tách biệt mối quan tâm:** Giúp tách biệt logic nghiệp vụ chính khỏi các tác vụ phụ trợ (side effects) như gửi email, cập nhật cache, hoặc ghi log audit.
- **Giảm thiểu lỗi:** Tránh các tình huống như gửi email xác nhận đơn hàng nhưng sau đó transaction bị rollback và đơn hàng không được lưu.
- **Dễ kiểm thử:** Các listener có thể được kiểm thử độc lập.

`TransactionalEventListener` là một công cụ mạnh mẽ để xây dựng các hệ thống phân tán và xử lý sự kiện dựa trên transaction một cách đáng tin cậy trong Spring Boot.

7.1.2. Spring's Transaction Abstraction (Trừu tượng hóa Transaction của Spring)

Một trong những điểm mạnh của Spring Framework là khả năng trừu tượng hóa (abstraction) các công nghệ quản lý transaction khác nhau. Điều này có nghĩa là bạn có thể viết code quản lý transaction một cách nhất quán, bất kể bạn đang sử dụng JDBC, JPA, Hibernate, JTA, hay bất kỳ công nghệ persistence nào khác. Spring sẽ tự động chuyển đổi các thao tác transaction của bạn thành các lệnh cụ thể cho công nghệ cơ bản.

7.1.2.1. Lợi ích của trừu tượng hóa

- **Tính di động (Portability):** Bạn có thể dễ dàng chuyển đổi giữa các công nghệ quản lý transaction (ví dụ: từ JDBC sang JPA) mà không cần thay đổi code nghiệp vụ liên quan đến transaction.
- **Tính nhất quán (Consistency):** Cung cấp một API quản lý transaction thống nhất, giúp giảm độ phức tạp và tăng tính nhất quán trong code.
- **Tách biệt mối quan tâm (Separation of Concerns):** Logic quản lý transaction được tách biệt khỏi logic nghiệp vụ, giúp code sạch sẽ và dễ bảo trì hơn.
- **Hỗ trợ Declarative Transaction Management:** Cho phép bạn quản lý transaction bằng annotation (`@Transactional`) thay vì phải viết code thủ công.

7.1.2.2. Các thành phần chính của trừu tượng hóa

Spring's transaction abstraction được xây dựng xung quanh các interface cốt lõi sau:

- **PlatformTransactionManager** : Đây là interface trung tâm trong Spring's transaction API. Nó định nghĩa các phương thức để bắt đầu, commit và rollback một transaction. Các triển khai cụ thể của interface này sẽ tương tác với các công nghệ transaction khác nhau:
 - `DataSourceTransactionManager` : Dành cho JDBC.
 - `JpaTransactionManager` : Dành cho JPA.
 - `HibernateTransactionManager` : Dành cho Hibernate (trước khi JPA trở nên phổ biến).
 - `JtaTransactionManager` : Dành cho JTA (Java Transaction API) để quản lý distributed transactions.
- **TransactionDefinition** : Interface này định nghĩa các thuộc tính của một transaction, bao gồm:
 - **Propagation Behavior (`Propagation`)**: Cách transaction lồng nhau hoạt động (ví dụ: `REQUIRED` , `REQUIRES_NEW`).
 - **Isolation Level (`Isolation`)**: Mức độ cô lập giữa các transaction đồng thời (ví dụ: `READ_COMMITTED` , `SERIALIZABLE`).

- **Timeout:** Thời gian tối đa mà transaction có thể chạy trước khi bị rollback tự động.
- **Read-only Status:** Cho biết transaction chỉ đọc hay có thể ghi. Transaction chỉ đọc có thể được tối ưu hóa bởi cơ sở dữ liệu.
- **Rollback Rules:** Các loại ngoại lệ nào sẽ kích hoạt rollback hoặc không kích hoạt rollback.
- **TransactionStatus** : Interface này cung cấp thông tin về trạng thái của một transaction đang hoạt động. Nó cho phép bạn kiểm tra xem transaction có bị đánh dấu là rollback-only hay không, hoặc liệu nó có phải là một transaction mới hay không.

7.1.2.3. Cách Spring Boot tự động cấu hình

Khi bạn thêm các dependency như `spring-boot-starter-data-jpa` hoặc `spring-boot-starter-jdbc`, Spring Boot sẽ tự động cấu hình một `PlatformTransactionManager` phù hợp (ví dụ: `JpaTransactionManager` cho JPA, `DataSourceTransactionManager` cho JDBC). Điều này giúp bạn không cần phải cấu hình thủ công `PlatformTransactionManager` trong hầu hết các trường hợp, làm cho việc bắt đầu với transaction management trở nên rất dễ dàng.

Nhờ có trừu tượng hóa này, bạn có thể tập trung vào logic nghiệp vụ của mình mà không cần lo lắng về chi tiết triển khai của việc quản lý transaction, đồng thời vẫn giữ được sự linh hoạt để thay đổi công nghệ persistence nếu cần.

7.8. Cách `@Transactional` hoạt động (Behind the Scenes)

Annotation `@Transactional` trong Spring hoạt động dựa trên cơ chế AOP (Aspect-Oriented Programming - Lập trình hướng khía cạnh). Khi bạn đánh dấu một lớp hoặc một phương thức bằng `@Transactional`, Spring sẽ tạo ra một proxy cho bean đó. Khi phương thức được gọi, proxy này sẽ chặn cuộc gọi và áp dụng logic quản lý transaction.

7.8.1. AOP Proxying

Spring sử dụng các AOP proxy để thêm hành vi transaction vào các bean của bạn mà không làm thay đổi code nghiệp vụ. Có hai loại proxy chính:

- **JDK Dynamic Proxies:** Đây là mặc định nếu bean của bạn triển khai ít nhất một interface. Proxy sẽ triển khai cùng interface đó và ủy quyền các cuộc gọi phương thức cho bean gốc.
- **CGLIB Proxies:** Nếu bean của bạn không triển khai bất kỳ interface nào, Spring sẽ sử dụng CGLIB để tạo một subclass của lớp bean của bạn. Proxy này sẽ ghi đè các phương thức được đánh dấu `@Transactional`.

Khi một phương thức được đánh dấu `@Transactional` được gọi, proxy sẽ chặn cuộc gọi và thực hiện các bước sau:

1. **Bắt đầu Transaction:** Proxy sẽ bắt đầu một transaction mới (hoặc tham gia vào một transaction hiện có tùy thuộc vào `Propagation` level).
2. **Thực thi phương thức nghiệp vụ:** Phương thức nghiệp vụ gốc của bạn được gọi.
3. **Xử lý kết quả:**
 - Nếu phương thức thực thi thành công và không có ngoại lệ nào được ném ra (hoặc chỉ các unchecked exception không được cấu hình để rollback), proxy sẽ **commit** transaction.
 - Nếu một `RuntimeException` (unchecked exception) hoặc một `Error` được ném ra, proxy sẽ **rollback** transaction. Bạn có thể tùy chỉnh hành vi này bằng cách sử dụng thuộc tính `rollbackFor` và `noRollbackFor` của `@Transactional`.
 - Nếu một `Checked Exception` được ném ra, transaction sẽ **không tự động rollback** (trừ khi bạn cấu hình `rollbackFor` cho nó).

7.8.2. Các vấn đề thường gặp với `@Transactional`

Do cơ chế proxying này, có một số trường hợp `@Transactional` có thể không hoạt động như mong đợi:

1. **Self-invocation (Tự gọi):** Nếu một phương thức `@Transactional` được gọi từ một phương thức khác **trong cùng một lớp** (sử dụng `this.methodName()`), transaction sẽ không được áp dụng. Điều này là do cuộc gọi không đi qua proxy mà gọi trực tiếp

phương thức gốc. Để khắc phục, bạn có thể inject bean của chính nó vào và gọi thông qua bean đã được inject, hoặc sử dụng `AopContext.currentProxy()` .

2. **Phương thức `private` hoặc `final` :** `@Transactional` không hoạt động trên các phương thức `private` hoặc `final` vì proxy không thể ghi đè hoặc chặn các phương thức này. Luôn sử dụng các phương thức `public` hoặc `protected` cho `@Transactional` .
3. **Checked Exceptions không tự động rollback:** Như đã đề cập, `@Transactional` mặc định chỉ rollback cho `RuntimeException` và `Error` . Nếu bạn muốn rollback cho một `Checked Exception` , bạn phải chỉ định nó trong thuộc tính `rollbackFor` .

Hiểu rõ cách `@Transactional` hoạt động giúp bạn tránh được các lỗi phổ biến và tận dụng tối đa khả năng quản lý transaction của Spring.

7.2. Cấu hình Transaction Management

Spring Boot làm cho việc cấu hình quản lý transaction trở nên rất đơn giản nhờ vào khả năng tự động cấu hình (auto-configuration). Tuy nhiên, hiểu rõ cách cấu hình hoạt động sẽ giúp bạn tùy chỉnh khi cần thiết.

7.2.1. Tự động cấu hình bởi Spring Boot

Khi bạn thêm các dependency liên quan đến cơ sở dữ liệu vào dự án Spring Boot của mình, Spring Boot sẽ tự động cấu hình các bean cần thiết cho quản lý transaction:

- **`spring-boot-starter-data-jpa`** : Khi dependency này có mặt, Spring Boot sẽ tự động cấu hình:
 - Một `DataSource` (nếu chưa có).
 - Một `EntityManagerFactory` (cho JPA).
 - Một `JpaTransactionManager` làm `PlatformTransactionManager` mặc định.
- **`spring-boot-starter-jdbc`** : Khi dependency này có mặt (và không có JPA), Spring Boot sẽ tự động cấu hình:
 - Một `DataSource` .

- Một `DataSourceTransactionManager` làm `PlatformTransactionManager` mặc định.

Điều này có nghĩa là trong hầu hết các trường hợp, bạn không cần phải khai báo bất kỳ bean `PlatformTransactionManager` nào một cách thủ công. Spring Boot sẽ lo liệu việc đó cho bạn.

7.2.2. Kích hoạt `@EnableTransactionManagement`

Trong các ứng dụng Spring truyền thống, bạn cần sử dụng `@EnableTransactionManagement` để kích hoạt tính năng quản lý transaction dựa trên annotation (`@Transactional`). Tuy nhiên, trong Spring Boot, annotation này thường không cần thiết vì nó đã được kích hoạt tự động thông qua auto-configuration khi bạn sử dụng các starter như JPA hoặc JDBC.

Tuy nhiên, nếu bạn đang sử dụng một công nghệ persistence không được Spring Boot tự động cấu hình transaction, hoặc bạn muốn kiểm soát chi tiết hơn, bạn có thể cần thêm `@EnableTransactionManagement` vào một lớp cấu hình của mình.

7.2.3. Cấu hình `PlatformTransactionManager` thủ công

Trong một số trường hợp, bạn có thể cần định nghĩa `PlatformTransactionManager` của riêng mình. Ví dụ:

- **Sử dụng nhiều `DataSource`:** Khi bạn có nhiều nguồn dữ liệu và muốn quản lý transaction riêng biệt cho từng nguồn.
- **Tùy chỉnh `PlatformTransactionManager`:** Khi bạn cần cấu hình các thuộc tính cụ thể cho transaction manager mà auto-configuration không cung cấp.
- **Sử dụng `JTA`:** Khi bạn cần quản lý distributed transactions trên nhiều nguồn tài nguyên (ví dụ: database và message queue) và cần một `JtaTransactionManager` .

Ví dụ cấu hình `JpaTransactionManager` thủ công:

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.orm.jpa.JpaTransactionManager;
import org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean;
import org.springframework.transaction.PlatformTransactionManager;
import
```

```

org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.persistence.EntityManagerFactory;
import javax.sql.DataSource;

@Configuration
@EnableTransactionManagement // Có thể cần nếu bạn tự định nghĩa TM
public class JpaConfig {

    // Giả sử bạn đã có DataSource và EntityManagerFactory được cấu hình
    // @Autowired private DataSource dataSource;
    // @Autowired private LocalContainerEntityManagerFactoryBean
    entityManagerFactory;

    @Bean
    public PlatformTransactionManager transactionManager(EntityManagerFactory
entityManagerFactory) {
        JpaTransactionManager transactionManager = new
JpaTransactionManager();
        transactionManager.setEntityManagerFactory(entityManagerFactory);
        return transactionManager;
    }
}

```

Ví dụ cấu hình DataSourceTransactionManager thủ công:

```

Java

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;
import
org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;

@Configuration
@EnableTransactionManagement
public class JdbcConfig {

    // Giả sử bạn đã có DataSource được cấu hình
    // @Autowired private DataSource dataSource;

    @Bean
    public PlatformTransactionManager transactionManager(DataSource
dataSource) {

```

```

        DataSourceTransactionManager transactionManager = new
DataSourceTransactionManager();
        transactionManager.setDataSource(dataSource);
        return transactionManager;
    }
}

```

Khi bạn định nghĩa một `PlatformTransactionManager` bean của riêng mình, Spring Boot sẽ không tự động cấu hình một cái khác, cho phép bạn toàn quyền kiểm soát.

7.2.4. Cấu hình cho Distributed Transactions (JTA)

Nếu bạn cần quản lý transaction trên nhiều nguồn tài nguyên (ví dụ: database và message queue) trong một môi trường phân tán, bạn sẽ cần sử dụng JTA (Java Transaction API).

Trong trường hợp này, bạn sẽ cấu hình một `JtaTransactionManager`.

Java

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.transaction.PlatformTransactionManager;
import
org.springframework.transaction.annotation.EnableTransactionManagement;
import org.springframework.transaction.jta.JtaTransactionManager;

@Configuration
@EnableTransactionManagement
public class JtaConfig {

    @Bean
    public PlatformTransactionManager transactionManager() {
        // Cần có một JTA Transaction Manager được cấu hình trong môi trường
ứng dụng server (ví dụ: WildFly, WebLogic)
        // Hoặc sử dụng một JTA implementation như Atomikos, Narayana
        return new JtaTransactionManager();
    }
}

```

Việc cấu hình transaction management trong Spring Boot thường rất đơn giản, nhưng việc hiểu rõ các tùy chọn cấu hình này sẽ giúp bạn xử lý các trường hợp phức tạp hơn một cách hiệu quả.

7.2.3. Các thuộc tính của @Transactional

Annotation `@Transactional` cung cấp một số thuộc tính để bạn có thể tùy chỉnh hành vi của transaction một cách chi tiết. Việc hiểu rõ các thuộc tính này là rất quan trọng để quản lý transaction hiệu quả.

- **value hoặc transactionManager** : (String) Tên của `PlatformTransactionManager` bean sẽ được sử dụng cho transaction này. Hữu ích khi bạn có nhiều transaction manager trong ứng dụng (ví dụ: cho nhiều DataSource).
- **Mặc định**: Spring sẽ tìm một `PlatformTransactionManager` bean duy nhất trong `ApplicationContext`.
- **propagation** : (Enum `Propagation`) Định nghĩa cách transaction lồng nhau hoạt động. Khi một phương thức được đánh dấu `@Transactional` được gọi từ một phương thức khác cũng có `@Transactional`, thuộc tính này sẽ quyết định xem có nên sử dụng transaction hiện có, tạo một transaction mới, hay thực hiện không có transaction.
- **REQUIRED (Mặc định)**: Nếu một transaction hiện có đang chạy, sử dụng nó. Nếu không, tạo một transaction mới.
- **SUPPORTS** : Nếu một transaction hiện có đang chạy, sử dụng nó. Nếu không, thực hiện không có transaction.
- **MANDATORY** : Nếu một transaction hiện có đang chạy, sử dụng nó. Nếu không, ném ra ngoại lệ.
- **REQUIRES_NEW** : Luôn tạo một transaction mới. Nếu một transaction hiện có đang chạy, nó sẽ bị tạm dừng cho đến khi transaction mới hoàn thành.
- **NOT_SUPPORTED** : Luôn thực hiện không có transaction. Nếu một transaction hiện có đang chạy, nó sẽ bị tạm dừng.
- **NEVER** : Luôn thực hiện không có transaction. Nếu một transaction hiện có đang chạy, ném ra ngoại lệ.
- **NESTED** : Nếu một transaction hiện có đang chạy, tạo một savepoint. Nếu không, hoạt động như **REQUIRED**. Rollback đến savepoint sẽ không rollback transaction bên

ngoài.

- **isolation** : (Enum `Isolation`) Định nghĩa mức độ cô lập của transaction. Mức độ cô lập càng cao thì dữ liệu càng nhất quán nhưng hiệu suất càng giảm do khóa (locking) nhiều hơn.
- **DEFAULT (Mặc định)**: Sử dụng mức độ cô lập mặc định của cơ sở dữ liệu.
- **READ_UNCOMMITTED** : Cho phép đọc dữ liệu chưa được commit bởi các transaction khác (dirty reads). Mức độ cô lập thấp nhất, hiệu suất cao nhất.
- **READ_COMMITTED** : Chỉ cho phép đọc dữ liệu đã được commit. Tránh dirty reads. (Mặc định trong PostgreSQL, SQL Server).
- **REPEATABLE_READ** : Đảm bảo rằng khi đọc cùng một dữ liệu nhiều lần trong cùng một transaction, kết quả sẽ nhất quán. Tránh dirty reads và non-repeatable reads. (Mặc định trong MySQL).
- **SERIALIZABLE** : Mức độ cô lập cao nhất. Đảm bảo rằng các transaction đồng thời được thực hiện tuần tự, tránh dirty reads, non-repeatable reads và phantom reads. Hiệu suất thấp nhất.
- **timeout** : (int) Thời gian tối đa (tính bằng giây) mà transaction có thể chạy trước khi bị rollback tự động. Giá trị mặc định là `-1` (không có timeout).
- **readOnly** : (boolean) Cho biết transaction chỉ đọc hay có thể ghi. Nếu đặt là `true` , cơ sở dữ liệu có thể tối ưu hóa hiệu suất bằng cách không cần duy trì các lock ghi. Nếu bạn cố gắng thực hiện thao tác ghi trong một transaction `readOnly` , một ngoại lệ có thể được ném ra.
- Mặc định: `false` .
- **rollbackFor** : (Class[] hoặc String[]) Một mảng các lớp `Throwable` (hoặc tên lớp đầy đủ) mà khi được ném ra, sẽ kích hoạt rollback transaction. Mặc định, chỉ `RuntimeException` và `Error` mới kích hoạt rollback.
- **rollbackForClassName** : (String[]) Tương tự như `rollbackFor` nhưng chấp nhận tên lớp dưới dạng String.

- **noRollbackFor** : (Class[] hoặc String[]) Một mảng các lớp **Throwable** (hoặc tên lớp đầy đủ) mà khi được ném ra, sẽ **không** kích hoạt rollback transaction. Hữu ích khi bạn muốn một số ngoại lệ cụ thể không làm rollback transaction.
- **noRollbackForClassName** : (String[]) Tương tự như **noRollbackFor** nhưng chấp nhận tên lớp dưới dạng String.

Ví dụ tổng hợp:

Java

```
@Transactional(
    value = "myCustomTransactionManager", // Sử dụng transaction manager có
    tên 'myCustomTransactionManager'
    propagation = Propagation.REQUIRED,
    isolation = Isolation.READ_COMMITTED,
    timeout = 60, // 60 giây
    readOnly = false,
    rollbackFor = {MyBusinessException.class, AnotherCheckedException.class},
    noRollbackFor = {ValidationException.class}
)
public void performComplexOperation() throws MyBusinessException,
AnotherCheckedException, ValidationException {
    // Logic nghiệp vụ phức tạp
}
```

Việc sử dụng đúng các thuộc tính này giúp bạn kiểm soát chặt chẽ hành vi của transaction, đảm bảo tính toàn vẹn dữ liệu và tối ưu hóa hiệu suất ứng dụng.

7.9. Rollback và Xử lý ngoại lệ trong Transaction

Một trong những khía cạnh quan trọng nhất của quản lý transaction là cách nó xử lý các ngoại lệ và quyết định khi nào nên rollback transaction.

7.9.1. Quy tắc Rollback mặc định của Spring

Theo mặc định, Spring Framework sẽ tự động rollback một transaction khi:

- Một **RuntimeException** (ngoại lệ không được kiểm tra - unchecked exception) được ném ra.

- Một `Error` được ném ra.

Spring sẽ **không** tự động rollback một transaction khi:

- Một `Checked Exception` (ngoại lệ được kiểm tra) được ném ra.

Lý do cho hành vi mặc định này là vì `Checked Exception` thường được coi là các điều kiện mà ứng dụng có thể phục hồi hoặc xử lý một cách duyên dáng, trong khi `RuntimeException` và `Error` thường chỉ ra một lỗi nghiêm trọng hơn, không thể phục hồi được, đòi hỏi phải hoàn tác toàn bộ thao tác.

7.9.2. Tùy chỉnh quy tắc Rollback với `@Transactional`

Bạn có thể tùy chỉnh hành vi rollback mặc định bằng cách sử dụng các thuộc tính `rollbackFor` và `noRollbackFor` của annotation `@Transactional`.

- **`rollbackFor`** : Chỉ định một hoặc nhiều loại ngoại lệ mà khi được ném ra, transaction sẽ bị rollback, bất kể đó là `Checked Exception` hay `RuntimeException`.
- **`noRollbackFor`** : Chỉ định một hoặc nhiều loại ngoại lệ mà khi được ném ra, transaction sẽ **không** bị rollback, ngay cả khi đó là `RuntimeException`.

7.9.3. Xử lý ngoại lệ trong phương thức `@Transactional`

Điều quan trọng là phải hiểu cách xử lý ngoại lệ bên trong một phương thức được đánh dấu `@Transactional`.

- **`try-catch` block bên trong `@Transactional`** :
Nếu bạn bắt một ngoại lệ bên trong phương thức `@Transactional` và không ném lại nó (hoặc ném lại một ngoại lệ khác không được cấu hình để rollback), transaction sẽ **không bị rollback**.
- **`TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()`** :
Nếu bạn muốn đánh dấu transaction là rollback-only từ bên trong một `try-catch` block mà không cần ném lại ngoại lệ, bạn có thể sử dụng phương thức này. Transaction sẽ bị rollback khi phương thức kết thúc.

7.9.4. `TransactionalEventListener`

Spring Framework 4.2 giới thiệu `@TransactionalEventListener` để lắng nghe các sự kiện được publish trong một transaction. Điều này rất hữu ích cho các tác vụ cần được thực hiện sau khi transaction đã hoàn tất (commit hoặc rollback).

- `@TransactionalEventListener` : Annotation này được đặt trên một phương thức lắng nghe sự kiện (event listener method) và cho phép bạn chỉ định `Phase` mà tại đó sự kiện sẽ được xử lý.
- `BEFORE_COMMIT` : Xử lý sự kiện ngay trước khi transaction được commit. Nếu listener ném ra ngoại lệ, transaction sẽ bị rollback.
- `AFTER_COMMIT` (**Mặc định**): Xử lý sự kiện sau khi transaction đã được commit thành công. Nếu listener ném ra ngoại lệ, transaction chính sẽ không bị ảnh hưởng.
- `AFTER_ROLLBACK` : Xử lý sự kiện sau khi transaction đã bị rollback.
- `AFTER_COMPLETION` : Xử lý sự kiện sau khi transaction đã hoàn tất (commit hoặc rollback).

`@TransactionalEventListener` là một công cụ mạnh mẽ để tách biệt các tác vụ phụ trợ (side effects) khỏi logic nghiệp vụ chính trong transaction, đảm bảo rằng các tác vụ này chỉ được thực hiện khi transaction chính đã hoàn tất theo một cách nhất định.

7.2. Spring's Transaction Abstraction (Trừu tượng hóa Transaction của Spring)

Một trong những điểm mạnh cốt lõi của Spring Framework là khả năng cung cấp một lớp trừu tượng hóa (abstraction layer) cho việc quản lý transaction. Điều này có nghĩa là bạn có thể viết mã quản lý transaction của mình mà không cần phải phụ thuộc trực tiếp vào các API quản lý transaction cụ thể (như JTA, JDBC, JPA, Hibernate). Spring sẽ tự động ánh xạ các thao tác transaction của bạn tới các API cơ bản phù hợp với công nghệ cơ sở dữ liệu hoặc ORM mà bạn đang sử dụng.

7.2.1. Lợi ích của Trừu tượng hóa Transaction

- **Độc lập với công nghệ:** Mã của bạn không bị ràng buộc với một công nghệ transaction cụ thể (ví dụ: JDBC `Connection.commit()` , JTA `UserTransaction`). Bạn có thể dễ dàng

chuyển đổi giữa các công nghệ transaction mà không cần thay đổi logic nghiệp vụ.

- **API nhất quán:** Spring cung cấp một API quản lý transaction thống nhất, đơn giản hóa việc phát triển và bảo trì.
- **Quản lý transaction khai báo (Declarative Transaction Management):** Đây là lợi ích lớn nhất. Với trừu tượng hóa, Spring có thể cung cấp `@Transactional` annotation, cho phép bạn quản lý transaction chỉ bằng cách đánh dấu các phương thức hoặc lớp, mà không cần viết mã quản lý transaction tường minh.
- **Tích hợp với Spring Data Access:** Spring Transaction Management tích hợp liền mạch với các module truy cập dữ liệu của Spring (JDBC, JPA, Hibernate, JDO, v.v.), đảm bảo rằng các transaction được quản lý đúng cách trong ngữ cảnh của các thao tác cơ sở dữ liệu.

7.2.2. Các thành phần chính của Spring Transaction Abstraction

1. `PlatformTransactionManager` Interface:

- Đây là interface trung tâm trong API quản lý transaction của Spring. Nó định nghĩa các phương thức cơ bản để bắt đầu, commit và rollback một transaction.
- Spring cung cấp các triển khai cụ thể của `PlatformTransactionManager` cho các công nghệ khác nhau:
 - `DataSourceTransactionManager` : Cho JDBC.
 - `JpaTransactionManager` : Cho JPA.
 - `HibernateTransactionManager` : Cho Hibernate (khi không dùng JPA).
 - `JtaTransactionManager` : Cho JTA (Java Transaction API), được sử dụng trong môi trường Java EE để quản lý transaction phân tán (distributed transactions).

2. `TransactionDefinition` Interface:

- Định nghĩa các thuộc tính của một transaction, bao gồm:
 - **Propagation Behavior (Hành vi lan truyền):** Cách một transaction mới liên kết với một transaction hiện có (ví dụ: `REQUIRED` , `REQUIRES_NEW`).

- **Isolation Level (Mức độ cô lập):** Mức độ bảo vệ dữ liệu khỏi các transaction đồng thời (ví dụ: `READ_COMMITTED` , `SERIALIZABLE`).
- **Timeout:** Thời gian tối đa mà transaction có thể chạy trước khi bị rollback tự động.
- **Read-only:** Chỉ định transaction chỉ đọc, có thể giúp tối ưu hóa hiệu suất.
- **Rollback Rules:** Các loại ngoại lệ nào sẽ kích hoạt rollback và loại nào không.

3. `TransactionStatus` Interface:

- Đại diện cho trạng thái của một transaction đang hoạt động. Nó cung cấp các phương thức để kiểm tra trạng thái của transaction (ví dụ: `isNewTransaction()` , `isRollbackOnly()`).

7.2.3. Cách Spring sử dụng trừu tượng hóa này

Khi bạn sử dụng `@Transactional` annotation, Spring sẽ sử dụng AOP (Aspect-Oriented Programming) để tạo ra một proxy cho bean của bạn. Khi một phương thức được đánh dấu `@Transactional` được gọi, proxy này sẽ chặn cuộc gọi và sử dụng `PlatformTransactionManager` đã được cấu hình để:

1. Bắt đầu một transaction mới (hoặc tham gia vào một transaction hiện có) dựa trên `TransactionDefinition` .
2. Thực thi phương thức nghiệp vụ của bạn.
3. Nếu phương thức hoàn thành mà không có ngoại lệ hoặc chỉ có các ngoại lệ được khai báo là không rollback, transaction sẽ được commit.
4. Nếu một ngoại lệ không được khai báo là không rollback xảy ra, transaction sẽ được rollback.

Nhờ lớp trừu tượng hóa này, bạn có thể tập trung vào logic nghiệp vụ của mình mà không cần lo lắng về chi tiết triển khai của việc quản lý transaction, giúp mã nguồn sạch sẽ, dễ đọc và dễ bảo trì hơn.

7.3. Cấu hình Transaction Management

Spring Boot làm cho việc cấu hình quản lý transaction trở nên rất đơn giản nhờ vào khả năng tự động cấu hình (auto-configuration). Khi bạn thêm các dependency liên quan đến cơ sở dữ liệu và JPA/JDBC, Spring Boot sẽ tự động phát hiện và cấu hình một `PlatformTransactionManager` phù hợp.

7.3.1. Auto-configuration của Spring Boot

Khi bạn thêm các starter dependency như `spring-boot-starter-data-jpa` (cho JPA/Hibernate) hoặc `spring-boot-starter-jdbc` (cho JDBC), Spring Boot sẽ tự động cấu hình:

- Một `DataSource` (nếu bạn đã cấu hình thông tin kết nối cơ sở dữ liệu trong `application.properties` hoặc `application.yml`).
- Một `EntityManagerFactory` (cho JPA).
- Một `PlatformTransactionManager` phù hợp:
 - `JpaTransactionManager` nếu bạn đang sử dụng JPA.
 - `DataSourceTransactionManager` nếu bạn đang sử dụng JDBC.

Điều này có nghĩa là trong hầu hết các trường hợp, bạn không cần phải viết bất kỳ cấu hình Java tường minh nào để kích hoạt transaction management. Chỉ cần thêm dependency và sử dụng `@Transactional`.

7.3.2. Kích hoạt Declarative Transaction Management

Trong các ứng dụng Spring truyền thống, bạn cần thêm `@EnableTransactionManagement` vào một lớp cấu hình để kích hoạt việc xử lý `@Transactional` annotation. Tuy nhiên, trong Spring Boot, annotation này được tự động áp dụng thông qua `spring-boot-autoconfigure` khi `spring-boot-starter-data-jpa` hoặc `spring-boot-starter-jdbc` có mặt trong classpath. Do đó, việc thêm `@EnableTransactionManagement` là không bắt buộc nhưng cũng không gây hại.

Java

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import
org.springframework.transaction.annotation.EnableTransactionManagement;
```

```

@SpringBootApplication
@EnableTransactionManagement // Thường không cần thiết trong Spring Boot vì
đã được auto-config
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}

```

7.3.3. Cấu hình PlatformTransactionManager tùy chỉnh

Mặc dù auto-configuration hoạt động tốt trong hầu hết các trường hợp, có những tình huống bạn cần cấu hình PlatformTransactionManager một cách tường minh. Ví dụ:

- Khi bạn có nhiều DataSource và cần các TransactionManager riêng biệt cho mỗi DataSource .
- Khi bạn muốn sử dụng một loại TransactionManager khác với loại mặc định được auto-configure.
- Khi bạn cần tùy chỉnh các thuộc tính của TransactionManager (ví dụ: defaultTimeout , nestedTransactionAllowed).

Ví dụ: Cấu hình JpaTransactionManager tường minh

Java

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.orm.jpa.JpaTransactionManager;
import org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean;
import org.springframework.transaction.PlatformTransactionManager;

import javax.sql.DataSource;

@Configuration
public class JpaConfig {

    // Giả sử bạn đã có DataSource và LocalContainerEntityManagerFactoryBean
    được cấu hình
    // @Autowired
    // private DataSource dataSource;

```

```

// @Autowired
// private LocalContainerEntityManagerFactoryBean entityManagerFactory;

@Bean
public PlatformTransactionManager
transactionManager(LocalContainerEntityManagerFactoryBean
entityManagerFactory) {
    JpaTransactionManager transactionManager = new
JpaTransactionManager();

transactionManager.setEntityManagerFactory(entityManagerFactory.getObject());
    // Bạn có thể tùy chỉnh thêm các thuộc tính khác nếu cần
    // transactionManager.setDefaultTimeout(60);
    return transactionManager;
}
}

```

Ví dụ: Cấu hình DataSourceTransactionManager tường minh

Java

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;

import javax.sql.DataSource;

@Configuration
public class JdbcConfig {

    // Giả sử bạn đã có DataSource được cấu hình
    // @Autowired
    // private DataSource dataSource;

    @Bean
    public PlatformTransactionManager transactionManager(DataSource
dataSource) {
        DataSourceTransactionManager transactionManager = new
DataSourceTransactionManager();
        transactionManager.setDataSource(dataSource);
        return transactionManager;
    }
}

```

7.3.4. Quản lý Transaction với Multiple DataSources

Khi bạn có nhiều nguồn dữ liệu (DataSource) trong ứng dụng, mỗi nguồn dữ liệu cần một PlatformTransactionManager riêng. Bạn cần đánh dấu một trong số chúng là @Primary nếu bạn muốn nó là transaction manager mặc định, hoặc chỉ định rõ tên bean của transaction manager trong @Transactional annotation.

Java

```
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import org.springframework.orm.jpa.JpaTransactionManager;
import org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean;
import org.springframework.transaction.PlatformTransactionManager;

import javax.persistence.EntityManagerFactory;

@Configuration
public class MultiDataSourceConfig {

    // Cấu hình DataSource và EntityManagerFactory cho primaryDataSource
    // ...

    // Cấu hình DataSource và EntityManagerFactory cho secondaryDataSource
    // ...

    @Primary
    @Bean(name = "primaryTransactionManager")
    public PlatformTransactionManager primaryTransactionManager(
        @Qualifier("primaryEntityManagerFactory") EntityManagerFactory
        entityManagerFactory) {
        return new JpaTransactionManager(entityManagerFactory);
    }

    @Bean(name = "secondaryTransactionManager")
    public PlatformTransactionManager secondaryTransactionManager(
        @Qualifier("secondaryEntityManagerFactory") EntityManagerFactory
        entityManagerFactory) {
        return new JpaTransactionManager(entityManagerFactory);
    }
}

// Sử dụng trong Service
@Service
```

```
public class MultiDatabaseService {  
  
    @Transactional("primaryTransactionManager")  
    public void saveToMainDatabase(Object entity) {  
        // ...  
    }  
  
    @Transactional("secondaryTransactionManager")  
    public void saveToAuditDatabase(Object entity) {  
        // ...  
    }  
}
```

Việc cấu hình transaction management trong Spring Boot rất linh hoạt, cho phép bạn dễ dàng thiết lập từ các trường hợp đơn giản nhất đến các kịch bản phức tạp với nhiều nguồn dữ liệu.

7.4. Các thuộc tính trong `@Transactional` Annotation

Annotation `@Transactional` không chỉ đơn thuần là bật/tắt transaction mà còn cho phép bạn tinh chỉnh hành vi của transaction thông qua một loạt các thuộc tính. Việc hiểu và sử dụng đúng các thuộc tính này là rất quan trọng để đảm bảo tính đúng đắn và hiệu suất của ứng dụng.

7.4.1. `propagation` (Mức độ lan truyền)

Thuộc tính `propagation` định nghĩa cách một transaction mới liên kết với một transaction hiện có (nếu có). Đây là một trong những thuộc tính quan trọng nhất để kiểm soát hành vi của transaction khi các phương thức `@Transactional` gọi lẫn nhau.

Các giá trị phổ biến:

- **`Propagation.REQUIRED` (Mặc định):**
 - Nếu một transaction đã tồn tại, phương thức sẽ tham gia vào transaction đó.
 - Nếu không có transaction nào tồn tại, một transaction mới sẽ được tạo.
 - Đây là lựa chọn phổ biến nhất và phù hợp cho hầu hết các trường hợp.

- **Propagation.SUPPORTS :**

- Nếu một transaction đã tồn tại, phương thức sẽ tham gia vào transaction đó.
- Nếu không có transaction nào tồn tại, phương thức sẽ được thực thi mà không có transaction.
- Thường được sử dụng cho các hoạt động chỉ đọc (read-only) không yêu cầu transaction.

- **Propagation.REQUIRES_NEW :**

- Luôn tạo một transaction mới. Nếu một transaction hiện tại đang tồn tại, nó sẽ bị tạm dừng cho đến khi transaction mới này hoàn thành.
- Hữu ích khi bạn muốn một phần của logic nghiệp vụ luôn chạy trong một transaction độc lập, không bị ảnh hưởng bởi transaction bên ngoài (ví dụ: ghi log audit).

- **Propagation.NOT_SUPPORTED :**

- Phương thức sẽ được thực thi mà không có transaction. Nếu một transaction hiện tại đang tồn tại, nó sẽ bị tạm dừng.
- Thường dùng cho các hoạt động không liên quan đến cơ sở dữ liệu hoặc các hoạt động mà bạn muốn đảm bảo không chạy trong transaction (ví dụ: gửi email).

- **Propagation.MANDATORY :**

- Yêu cầu một transaction hiện tại phải tồn tại. Nếu không có transaction nào, một `TransactionRequiredException` sẽ được ném ra.
- Đảm bảo rằng phương thức này luôn được gọi trong một ngữ cảnh transaction.

- **Propagation.NEVER :**

- Yêu cầu không có transaction nào tồn tại. Nếu một transaction hiện tại đang tồn tại, một `IllegalTransactionStateException` sẽ được ném ra.
- Đảm bảo rằng phương thức này không bao giờ được gọi trong một ngữ cảnh transaction.

- **Propagation.NESTED :**

- Nếu một transaction đã tồn tại, nó sẽ tạo một savepoint (điểm lưu) và thực thi trong một transaction lồng nhau. Nếu transaction lồng nhau này thất bại, nó có thể rollback về savepoint mà không ảnh hưởng đến transaction bên ngoài.
- Chỉ hoạt động với các `PlatformTransactionManager` hỗ trợ nested transactions (ví dụ: `DataSourceTransactionManager` với JDBC 3.0+).

7.4.2. **isolation** (Mức độ cô lập)

Thuộc tính `isolation` định nghĩa mức độ cô lập giữa các transaction đồng thời. Mức độ cô lập cao hơn sẽ đảm bảo tính nhất quán dữ liệu tốt hơn nhưng có thể làm giảm hiệu suất do tăng cường khóa (locking).

Các mức độ cô lập (từ thấp đến cao):

- **Isolation.READ_UNCOMMITTED :**

- Cho phép đọc dữ liệu chưa được commit bởi các transaction khác (dirty reads).
- Mức độ cô lập thấp nhất, hiệu suất cao nhất, nhưng rủi ro về tính nhất quán dữ liệu cao.

- **Isolation.READ_COMMITTED (Mặc định trong hầu hết RDBMS):**

- Chỉ cho phép đọc dữ liệu đã được commit.
- Tránh dirty reads, nhưng có thể xảy ra non-repeatable reads (đọc cùng một hàng hai lần nhưng nhận được giá trị khác nhau) và phantom reads (đọc cùng một truy vấn hai lần nhưng nhận được số lượng hàng khác nhau).

- **Isolation.REPEATABLE_READ :**

- Đảm bảo rằng nếu bạn đọc một hàng nhiều lần trong cùng một transaction, bạn sẽ luôn nhận được cùng một giá trị.
- Tránh dirty reads và non-repeatable reads, nhưng vẫn có thể xảy ra phantom reads.

- **Isolation.SERIALIZABLE :**

- Mức độ cô lập cao nhất. Đảm bảo rằng các transaction được thực thi tuần tự, như thể chúng không đồng thời.
- Tránh tất cả các vấn đề đồng thời (dirty reads, non-repeatable reads, phantom reads), nhưng hiệu suất thấp nhất do khóa nhiều.

7.4.3. `readOnly` (Chỉ đọc)

- **`boolean readOnly()`** : Mặc định là `false` . Nếu đặt là `true` , transaction sẽ được tối ưu hóa cho các hoạt động chỉ đọc. Điều này có thể giúp cải thiện hiệu suất vì cơ sở dữ liệu có thể bỏ qua việc quản lý các khóa ghi (write locks).
- **Lưu ý:** Thuộc tính này chỉ là một gợi ý cho `PlatformTransactionManager` và không phải tất cả các cơ sở dữ liệu hoặc ORM đều hỗ trợ hoặc tận dụng được nó một cách hiệu quả.

7.4.4. `timeout` (Thời gian chờ)

- **`int timeout()`** : Mặc định là `-1` (không có timeout). Đặt thời gian tối đa (tính bằng giây) mà transaction có thể chạy trước khi bị rollback tự động. Nếu transaction vượt quá thời gian này, một `TransactionTimeoutException` sẽ được ném ra.
- Hữu ích để ngăn chặn các transaction chạy quá lâu và gây tắc nghẽn tài nguyên.

7.4.5. `rollbackFor` và `noRollbackFor` (Quy tắc Rollback)

Các thuộc tính này cho phép bạn định nghĩa các loại ngoại lệ nào sẽ kích hoạt rollback và loại nào sẽ không.

- **`Class<? extends Throwable>[] rollbackFor()`** :
 - Mặc định, Spring sẽ rollback transaction cho các `RuntimeException` và `Error` .
 - Sử dụng thuộc tính này để chỉ định thêm các loại `Checked Exception` mà bạn muốn kích hoạt rollback.
 - **Ví dụ:** `rollbackFor = {MyCheckedException.class}`
- **`Class<? extends Throwable>[] noRollbackFor()`** :

- Sử dụng thuộc tính này để chỉ định các loại `RuntimeException` hoặc `Error` mà bạn **không muốn** kích hoạt rollback.
- **Ví dụ:** `noRollbackFor = {IllegalArgumentException.class}` (nếu bạn muốn một `IllegalArgumentException` không làm rollback transaction).

Ví dụ tổng hợp:

Java

```
import org.springframework.transaction.annotation.Isolation;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Service
public class AdvancedOrderService {

    @Transactional(
        propagation = Propagation.REQUIRED,
        isolation = Isolation.READ_COMMITTED,
        readOnly = false,
        timeout = 60, // 60 giây
        rollbackFor = {OrderProcessingException.class}, // Rollback nếu
        OrderProcessingException xảy ra
        noRollbackFor = {InvalidInputException.class} // Không rollback nếu
        InvalidInputException xảy ra
    )
    public void processComplexOrder(Order order) throws
    OrderProcessingException, InvalidInputException {
        // Logic xử lý đơn hàng phức tạp
        if (order.getTotalAmount() < 0) {
            throw new InvalidInputException("Total amount cannot be
            negative"); // Sẽ không rollback
        }
        // ...
        if (!inventoryService.checkStock(order.getProductId(),
        order.getQuantity())) {
            throw new OrderProcessingException("Insufficient stock"); // Sẽ
            rollback
        }
        // ...
    }

    @Transactional(readOnly = true, isolation = Isolation.REPEATABLE_READ)
    public Order getOrderDetails(Long orderId) {
        // Logic lấy chi tiết đơn hàng, đảm bảo tính nhất quán khi đọc
    }
}
```

```
        return orderRepository.findById(orderId).orElse(null);
    }
}
```

Việc nắm vững các thuộc tính này cho phép bạn kiểm soát chính xác hành vi của transaction, từ đó xây dựng các ứng dụng mạnh mẽ, hiệu quả và đáng tin cậy hơn.

7.5. How does `@Transactional` Work? (Cách `@Transactional` hoạt động)

Annotation `@Transactional` là một trong những tính năng mạnh mẽ nhất của Spring Framework, cho phép quản lý transaction một cách khai báo (declarative transaction management). Thay vì phải viết mã quản lý transaction tường minh (như `try-catch-commit-rollback` với `PlatformTransactionManager`), bạn chỉ cần đánh dấu các phương thức hoặc lớp bằng `@Transactional`, và Spring sẽ tự động xử lý các khía cạnh liên quan đến transaction.

Vậy, làm thế nào mà một annotation đơn giản lại có thể thực hiện được điều này?

7.5.1. Cơ chế AOP (Aspect-Oriented Programming)

Cơ chế đằng sau `@Transactional` là **Aspect-Oriented Programming (AOP)**. Spring sử dụng AOP để tạo ra các proxy cho các bean được đánh dấu `@Transactional`. Khi một phương thức được gọi trên một bean được proxy, proxy sẽ chặn cuộc gọi đó và thêm vào logic quản lý transaction trước và sau khi phương thức thực sự được thực thi.

Các bước hoạt động:

- 1. Bean Post-Processing:** Khi Spring IoC container khởi tạo các bean, nó sẽ kiểm tra xem có bean nào được đánh dấu `@Transactional` hay không. Nếu có, Spring sẽ sử dụng một `BeanPostProcessor` (cụ thể là `AnnotationAwareAspectJAutoProxyCreator` hoặc tương tự) để tạo ra một proxy cho bean đó.
- 2. Proxy Creation:** Proxy này là một đối tượng bao bọc (wrapper) xung quanh bean gốc. Nó có cùng giao diện (interface) với bean gốc (nếu bean gốc implement interface) hoặc là một subclass của bean gốc (nếu bean gốc là một lớp cụ thể và không implement interface, sử dụng CGLIB).

3. **Method Interception:** Khi một phương thức được đánh dấu `@Transactional` (hoặc một phương thức trong một lớp được đánh dấu `@Transactional`) được gọi, cuộc gọi đó sẽ không trực tiếp đến bean gốc mà sẽ bị chặn bởi proxy.
4. **Transaction Aspect Logic:** Proxy sẽ ủy quyền cuộc gọi đến một aspect (cụ thể là `TransactionInterceptor`). `TransactionInterceptor` này chứa logic để:
- **Bắt đầu Transaction:** Trước khi phương thức nghiệp vụ được gọi, `TransactionInterceptor` sẽ sử dụng `PlatformTransactionManager` đã được cấu hình để bắt đầu một transaction mới (hoặc tham gia vào một transaction hiện có) dựa trên các thuộc tính của `@Transactional` (propagation, isolation, timeout, readOnly).
 - **Thực thi phương thức nghiệp vụ:** Sau khi transaction được bắt đầu, phương thức nghiệp vụ thực sự của bean gốc sẽ được gọi.
 - **Commit/Rollback Transaction:**
 - Nếu phương thức nghiệp vụ hoàn thành mà không ném ra bất kỳ ngoại lệ nào (hoặc chỉ ném ra các `Checked Exception` không được cấu hình để rollback), `TransactionInterceptor` sẽ yêu cầu `PlatformTransactionManager` **commit** transaction.
 - Nếu phương thức nghiệp vụ ném ra một `RuntimeException` hoặc một `Error` (hoặc một `Checked Exception` được cấu hình trong `rollbackFor`), `TransactionInterceptor` sẽ yêu cầu `PlatformTransactionManager` **rollback** transaction.
5. **Return Result:** Sau khi transaction được xử lý (commit hoặc rollback), kết quả của phương thức nghiệp vụ (hoặc ngoại lệ) sẽ được trả về cho người gọi ban đầu.

7.5.2. Các điểm quan trọng cần lưu ý

- **Chỉ hoạt động trên các phương thức public:** Do cơ chế proxy của Spring AOP, `@Transactional` chỉ hoạt động trên các phương thức `public`. Nếu bạn đặt `@Transactional` trên một phương thức `private` hoặc `protected`, nó sẽ bị bỏ qua.
- **Self-invocation (Tự gọi):** Nếu một phương thức `@Transactional` được gọi từ một phương thức khác **trong cùng một lớp** (self-invocation), transaction sẽ không được áp dụng. Điều này là do cuộc gọi `this.myTransactionalMethod()` không đi qua proxy mà gọi

trực tiếp đến bean gốc. Để khắc phục, bạn có thể inject bean của chính nó vào và gọi thông qua bean đã được inject, hoặc sử dụng `AopContext.currentProxy()` .

- **Checked vs. Unchecked Exceptions:**

- Mặc định, Spring chỉ rollback transaction cho `RuntimeException` (unchecked exceptions) và `Error` .
- Nó **không** rollback cho `Checked Exception` (các ngoại lệ mà bạn phải khai báo trong chữ ký phương thức hoặc bắt). Nếu bạn muốn rollback cho một `Checked Exception` , bạn phải chỉ định nó trong thuộc tính `rollbackFor` của `@Transactional` .
- **Transaction Synchronization:** Spring cũng cung cấp cơ chế đồng bộ hóa transaction, cho phép các tài nguyên khác (như Hibernate Session, JPA EntityManager) được liên kết với transaction hiện tại. Điều này đảm bảo rằng tất cả các thao tác cơ sở dữ liệu trong cùng một transaction đều hoạt động trên cùng một ngữ cảnh và được commit/rollback cùng nhau.

Hiểu rõ cách `@Transactional` hoạt động dưới lớp vỏ bọc của AOP giúp bạn sử dụng nó hiệu quả hơn và gỡ lỗi các vấn đề liên quan đến transaction một cách chính xác.

7.8. Rollback & Handle Exception (Lý thuyết chuyên sâu)

Việc hiểu rõ cơ chế rollback và xử lý ngoại lệ trong Spring Transaction Management là cực kỳ quan trọng để đảm bảo tính toàn vẹn dữ liệu và xây dựng các ứng dụng đáng tin cậy. Spring cung cấp các cơ chế linh hoạt để kiểm soát khi nào một transaction nên được rollback và khi nào nó nên được commit, ngay cả khi có ngoại lệ xảy ra.

7.8.1. Cơ chế Rollback mặc định của Spring

Theo mặc định, Spring Transaction Management tuân theo các quy tắc sau:

- **Unchecked Exceptions (RuntimeException và các subclass):** Sẽ gây ra rollback tự động. Điều này bao gồm các ngoại lệ như `NullPointerException` , `IllegalArgumentException` , `DataAccessException` , và tất cả các custom exception kế thừa từ `RuntimeException` .

- **Checked Exceptions (Exception và các subclass không phải RuntimeException):** Sẽ KHÔNG gây ra rollback tự động. Điều này có nghĩa là transaction sẽ được commit ngay cả khi có checked exception xảy ra, trừ khi bạn cấu hình khác.
- **Error (các lỗi hệ thống nghiêm trọng):** Sẽ gây ra rollback tự động.

Lý do cho hành vi này là Spring giả định rằng checked exceptions thường đại diện cho các tình huống nghiệp vụ có thể phục hồi được (recoverable business conditions), trong khi unchecked exceptions thường đại diện cho các lỗi lập trình hoặc lỗi hệ thống không thể phục hồi.

Ví dụ minh họa hành vi mặc định:

Java

```
@Service
public class DefaultRollbackBehaviorService {

    @Transactional
    public void methodWithRuntimeException() {
        userRepository.save(new User("John"));

        // RuntimeException sẽ gây rollback - User "John" sẽ không được lưu
        throw new IllegalArgumentException("This will cause rollback");
    }

    @Transactional
    public void methodWithCheckedException() throws IOException {
        userRepository.save(new User("Jane"));

        // Checked exception KHÔNG gây rollback - User "Jane" sẽ được lưu
        throw new IOException("This will NOT cause rollback");
    }

    @Transactional
    public void methodWithError() {
        userRepository.save(new User("Bob"));

        // Error sẽ gây rollback - User "Bob" sẽ không được lưu
        throw new OutOfMemoryError("This will cause rollback");
    }
}
```

7.8.2. Tùy chỉnh hành vi Rollback với `rollbackFor` và `noRollbackFor`

Spring cho phép bạn tùy chỉnh hành vi rollback thông qua các thuộc tính `rollbackFor` và `noRollbackFor` của annotation `@Transactional`.

`rollbackFor` : Chỉ định các exception class mà transaction nên rollback, ngay cả khi chúng là checked exceptions.

`noRollbackFor` : Chỉ định các exception class mà transaction KHÔNG nên rollback, ngay cả khi chúng là unchecked exceptions.

Java

```
@Service
public class CustomRollbackBehaviorService {

    // Rollback cho cả checked và unchecked exceptions
    @Transactional(rollbackFor = {IOException.class, SQLException.class,
    BusinessException.class})
    public void methodWithCustomRollbackRules() throws IOException,
    SQLException {
        userRepository.save(new User("Alice"));

        // Bây giờ IOException sẽ gây rollback
        if (someCondition) {
            throw new IOException("This will now cause rollback");
        }

        // SQLException cũng sẽ gây rollback
        if (anotherCondition) {
            throw new SQLException("This will also cause rollback");
        }
    }

    // Không rollback cho một số unchecked exceptions cụ thể
    @Transactional(noRollbackFor = {IllegalArgumentException.class,
    ValidationException.class})
    public void methodWithNoRollbackRules() {
        userRepository.save(new User("Charlie"));

        // IllegalArgumentException sẽ KHÔNG gây rollback
        if (invalidInput) {
            throw new IllegalArgumentException("This will NOT cause
rollback");
        }
    }
}
```



```

        // Nhưng các RuntimeException khác vẫn sẽ gây rollback
        if (otherError) {
            throw new NullPointerException("This will still cause rollback");
        }
    }

    // Kết hợp cả hai thuộc tính
    @Transactional(
        rollbackFor = {IOException.class, SQLException.class},
        noRollbackFor = {IllegalArgumentException.class}
    )
    public void methodWithMixedRules() throws IOException, SQLException {
        // IOException và SQLException sẽ gây rollback
        // IllegalArgumentException sẽ KHÔNG gây rollback
        // Các RuntimeException khác vẫn sẽ gây rollback theo mặc định
    }
}

```

7.8.3. Sử dụng Exception Hierarchy để kiểm soát Rollback

Bạn có thể tận dụng tính kế thừa của exception để tạo ra một hệ thống phân loại exception có cấu trúc, giúp kiểm soát rollback một cách tinh tế.

Java

```

// Base exception cho các lỗi nghiệp vụ không cần rollback
public abstract class BusinessException extends Exception {
    public BusinessException(String message) {
        super(message);
    }
}

// Các lỗi nghiệp vụ có thể phục hồi
public class ValidationException extends BusinessException {
    public ValidationException(String message) {
        super(message);
    }
}

public class InsufficientStockException extends BusinessException {
    public InsufficientStockException(String message) {
        super(message);
    }
}

```

```

// Base exception cho các lỗi nghiệp vụ cần rollback
public abstract class CriticalBusinessException extends RuntimeException {
    public CriticalBusinessException(String message) {
        super(message);
    }
}

public class PaymentFailedException extends CriticalBusinessException {
    public PaymentFailedException(String message) {
        super(message);
    }
}

public class DataCorruptionException extends CriticalBusinessException {
    public DataCorruptionException(String message) {
        super(message);
    }
}

@Service
public class StructuredExceptionService {

    // Chỉ rollback cho critical business exceptions
    @Transactional(rollbackFor = {CriticalBusinessException.class})
    public void processOrder(Order order) throws BusinessException {
        validateOrder(order); // Có thể ném ValidationException (không
rollback)

        try {
            processPayment(order); // Có thể ném PaymentFailedException
(rollback)
            updateInventory(order);

        } catch (PaymentFailedException e) {
            // Log lỗi và re-throw để rollback
            logger.error("Payment failed for order {}: {}", order.getId(),
e.getMessage());
            throw e; // Transaction sẽ rollback
        } catch (ValidationException e) {
            // Log cảnh báo nhưng không rollback
            logger.warn("Validation failed for order {}: {}", order.getId(),
e.getMessage());
            throw e; // Transaction sẽ KHÔNG rollback
        }
    }
}

```

7.8.4. Programmatic Rollback với `TransactionStatus`

Trong một số trường hợp, bạn có thể cần rollback transaction một cách thủ công dựa trên logic nghiệp vụ phức tạp, không chỉ dựa vào exception.

Java

```
@Service
public class ProgrammaticRollbackService {

    private final PlatformTransactionManager transactionManager;

    public ProgrammaticRollbackService(PlatformTransactionManager
transactionManager) {
        this.transactionManager = transactionManager;
    }

    public OrderResult processOrderWithManualControl(Order order) {
        DefaultTransactionDefinition def = new
DefaultTransactionDefinition();
        TransactionStatus status = transactionManager.getTransaction(def);

        try {
            // Thực hiện các thao tác nghiệp vụ
            order.setStatus("PROCESSING");
            orderRepository.save(order);

            PaymentResult paymentResult =
paymentService.processPayment(order);

            if (paymentResult.isSuccessful()) {
                order.setStatus("PAID");
                orderRepository.save(order);

                // Commit thủ công
                transactionManager.commit(status);
                return OrderResult.success(order);

            } else if (paymentResult.isRetryable()) {
                // Rollback nhưng không ném exception
                transactionManager.rollback(status);
                return
OrderResult.retryLater(paymentResult.getErrorMessage());

            } else {
                // Rollback và trả về lỗi
                transactionManager.rollback(status);
            }
        }
    }
}
```

```

        return OrderResult.failed(paymentResult.getErrorMessage());
    }

    } catch (Exception e) {
        // Rollback trong trường hợp có exception
        transactionManager.rollback(status);
        return OrderResult.error(e.getMessage());
    }
}

// Sử dụng TransactionTemplate cho cách tiếp cận functional
@Autowired
private TransactionTemplate transactionTemplate;

public OrderResult processOrderWithTemplate(Order order) {
    return transactionTemplate.execute(status -> {
        try {
            order.setStatus("PROCESSING");
            orderRepository.save(order);

            PaymentResult paymentResult =
paymentService.processPayment(order);

            if (!paymentResult.isSuccessful()) {
                // Đánh dấu rollback mà không ném exception
                status.setRollbackOnly();
                return
OrderResult.failed(paymentResult.getErrorMessage());
            }

            order.setStatus("PAID");
            orderRepository.save(order);
            return OrderResult.success(order);

        } catch (Exception e) {
            // Exception sẽ tự động gây rollback
            throw new RuntimeException("Order processing failed", e);
        }
    });
}
}

```

7.8.5. Nested Transactions và Rollback Behavior

Khi có các transaction lồng nhau (nested transactions), hành vi rollback có thể trở nên phức tạp, đặc biệt với các propagation level khác nhau.

Java

```
@Service
public class NestedTransactionRollbackService {

    private final AuditService auditService;
    private final NotificationService notificationService;

    @Transactional
    public void processOrderWithNesting(Order order) {
        try {
            // Transaction chính
            order.setStatus("PROCESSING");
            orderRepository.save(order);

            // Gọi service khác với REQUIRES_NEW
            auditService.logOrderStart(order.getId()); // Transaction riêng

            // Xử lý payment
            processPayment(order); // Có thể ném exception

            order.setStatus("COMPLETED");
            orderRepository.save(order);

            // Gọi notification với REQUIRES_NEW
            notificationService.sendOrderConfirmation(order.getId()); //
            Transaction riêng

        } catch (PaymentException e) {
            // Transaction chính sẽ rollback
            // Nhưng audit log và notification (nếu đã thực hiện) sẽ được giữ
            lại
            // vì chúng sử dụng REQUIRES_NEW

            logger.error("Order processing failed: {}", e.getMessage());
            throw e; // Re-throw để rollback transaction chính
        }
    }
}

@Service
public class AuditService {

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void logOrderStart(Long orderId) {
        // Transaction này độc lập, sẽ không bị rollback khi transaction
        chính thất bại
        AuditLog log = new AuditLog("ORDER_START", orderId,
```

```

        LocalDateTime.now());
        auditLogRepository.save(log);
    }
}

@Service
public class NotificationService {

    @Transactional(propagation = Propagation.REQUIRES_NEW,
        noRollbackFor = {NotificationException.class})
    public void sendOrderConfirmation(Long orderId) {
        try {
            // Gửi email xác nhận
            emailService.sendConfirmation(orderId);

            // Log thành công
            NotificationLog log = new NotificationLog(orderId, "EMAIL_SENT",
LocalDateTime.now());
            notificationLogRepository.save(log);

        } catch (NotificationException e) {
            // Vẫn log lỗi nhưng không rollback transaction
            NotificationLog errorLog = new NotificationLog(orderId,
"EMAIL_FAILED", LocalDateTime.now());
            errorLog.setErrorMessage(e.getMessage());
            notificationLogRepository.save(errorLog);

            // Không re-throw exception để tránh ảnh hưởng đến transaction
chính
            logger.warn("Failed to send notification for order {}: {}",
orderId, e.getMessage());
        }
    }
}

```

7.8.6. Best Practices cho Exception Handling trong Transactions

1. Phân loại Exception rõ ràng:

- Sử dụng checked exceptions cho các lỗi nghiệp vụ có thể phục hồi.
- Sử dụng unchecked exceptions cho các lỗi hệ thống hoặc lỗi nghiệp vụ nghiêm trọng.
- Tạo hierarchy exception có cấu trúc để dễ quản lý rollback rules.

2. Cấu hình Rollback Rules phù hợp:

- Sử dụng `rollbackFor` để rollback cho các checked exceptions quan trọng.
- Sử dụng `noRollbackFor` cho các unchecked exceptions không cần rollback.
- Cân nhắc sử dụng base exception classes để áp dụng rules cho nhiều exception cùng lúc.

3. Xử lý Nested Transactions cẩn thận:

- Hiểu rõ propagation behavior và tác động của nó đến rollback.
- Sử dụng `REQUIRES_NEW` cho các operations độc lập (logging, auditing).
- Cẩn thận với `NESTED` propagation vì không phải tất cả database đều hỗ trợ.

4. Logging và Monitoring:

- Luôn log các transaction rollback để dễ debug.
- Monitor tỷ lệ rollback để phát hiện vấn đề hiệu suất hoặc logic.
- Sử dụng transaction events để hook vào lifecycle của transaction.

Việc nắm vững các khái niệm này sẽ giúp bạn xây dựng các ứng dụng Spring Boot với transaction management mạnh mẽ, đáng tin cậy và dễ bảo trì.